

Multiprocessor Caches

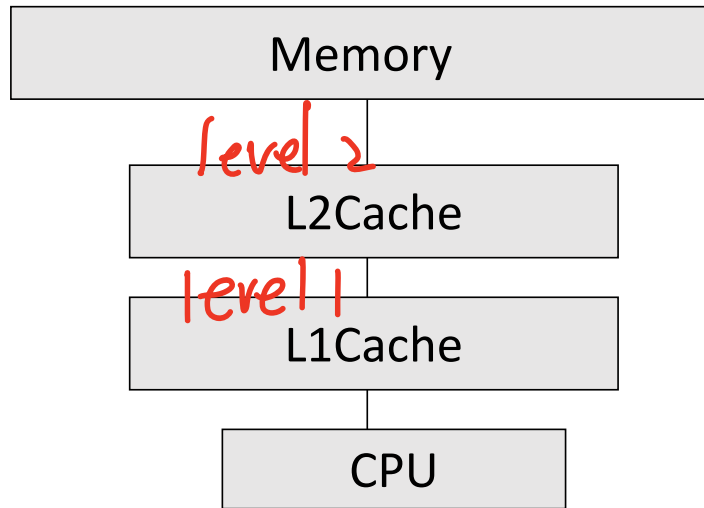
mano@auckland.ac.nz

Caches

- Caches bridge the speed gap between the processor and the main memory.
- Small fast memory holding data that is likely to be accessed in the near future.

Hardware Architecture

- A single-processor machine has: a CPU, memory, cache (primary & secondary), disk, and other peripheral devices.



Recapitulate:

Locality

局限性

Cache lines

Hit/miss/miss penalty

Write through/write back

Direct mapped

Set associative

Replacement policies

Locality

- Caches rely on two properties of the access patterns of most programs: 时空局部性
 - **temporal locality** - if a memory location is accessed once, it is likely to be accessed again soon
 - **spatial locality** - if one memory location is accessed then nearby memory locations are also likely to be accessed.

Cache Lines

- In order to exploit spatial locality, caches operate on several words at a time, called a *cache line* or a *cache block*.
 - Main memory reads and writes are whole cache lines.
 - This takes advantage of the principle of spatial locality of reference: if one location is read then nearby locations (particularly following locations) are likely to be read soon afterwards.

mult words.

外罰

Hits, Misses and Miss Penalty

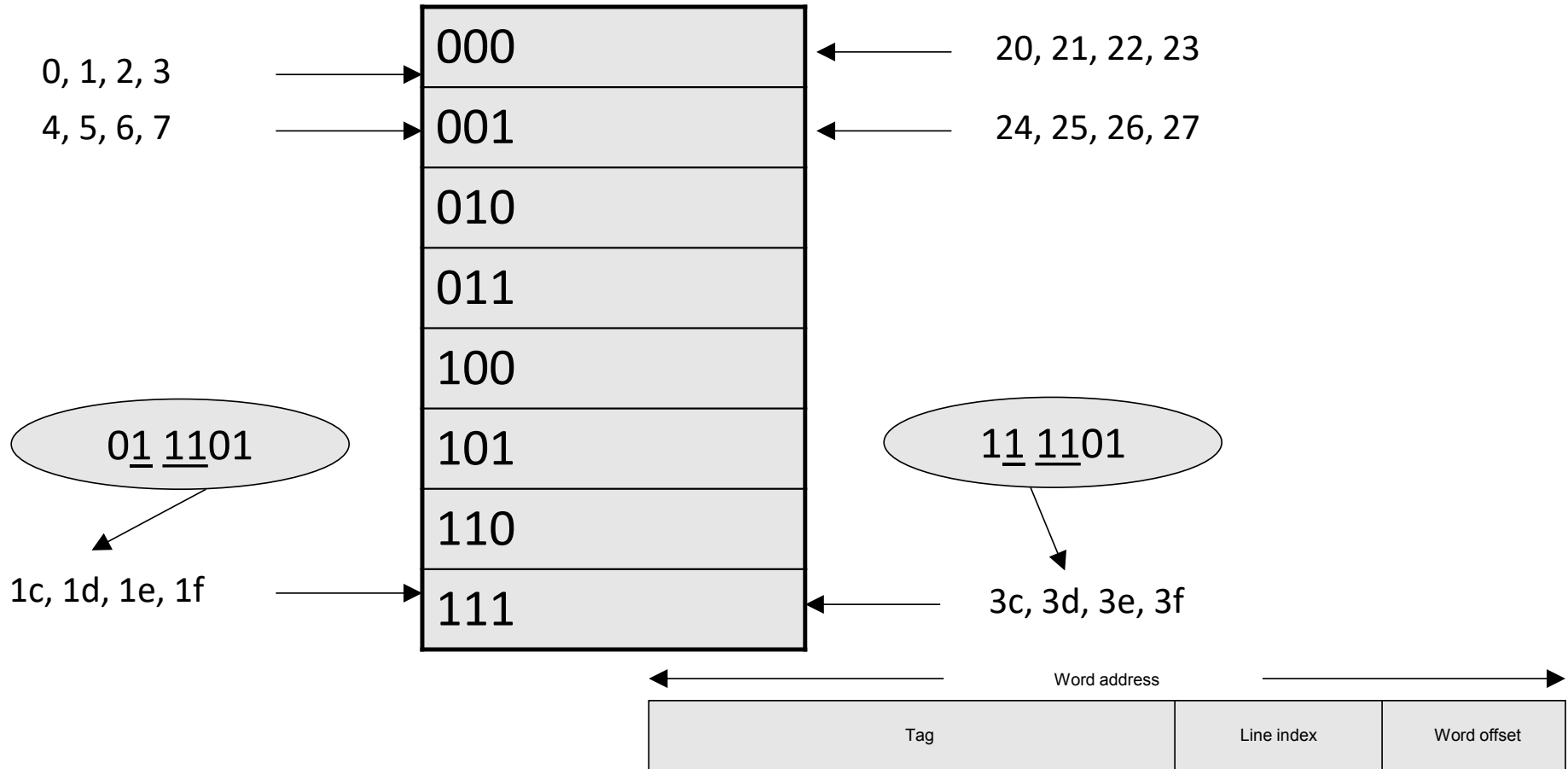
- If the data being accessed is in the cache, then we have a *cache hit*.
- If the data is not cached (a *cache miss*) then it is fetched from main memory and also saved in the cache.
- The time it takes to service a memory request under a cache miss is called a *miss penalty*.

Write through and write back

- When writing, a *write-through* policy updates both cache and main memory
- A *write-back* policy only updates the cache, and written to the main memory only when required.

only value require.

Direct-Mapped Caches



Categorizing cache misses

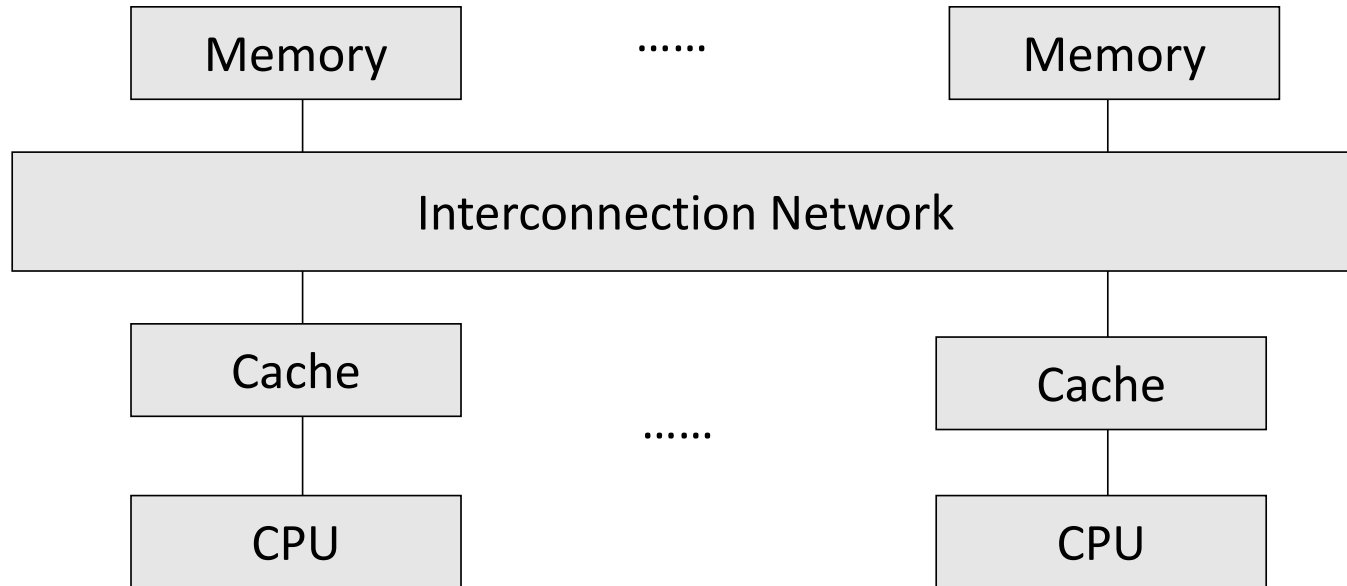
- ✓ • **Compulsory miss:** First ever reference to a cache line. *Can be reduced.*
- **Capacity miss:** Miss due to the cache not being big enough to hold the current data set. If the number of active lines is more than the cache can contain, capacity misses take place.

Categorizing cache misses

- ✓ • **Conflict miss:** The required line was previously there in the cache, but was replaced by another line which happened to map to the same cache location.
 - Conflict misses take place because of limited or zero associativity when lines must be discarded in order to accommodate new lines which are mapped to the same line in the cache.
 - A miss occurs when the replaced line needs to be accessed again.

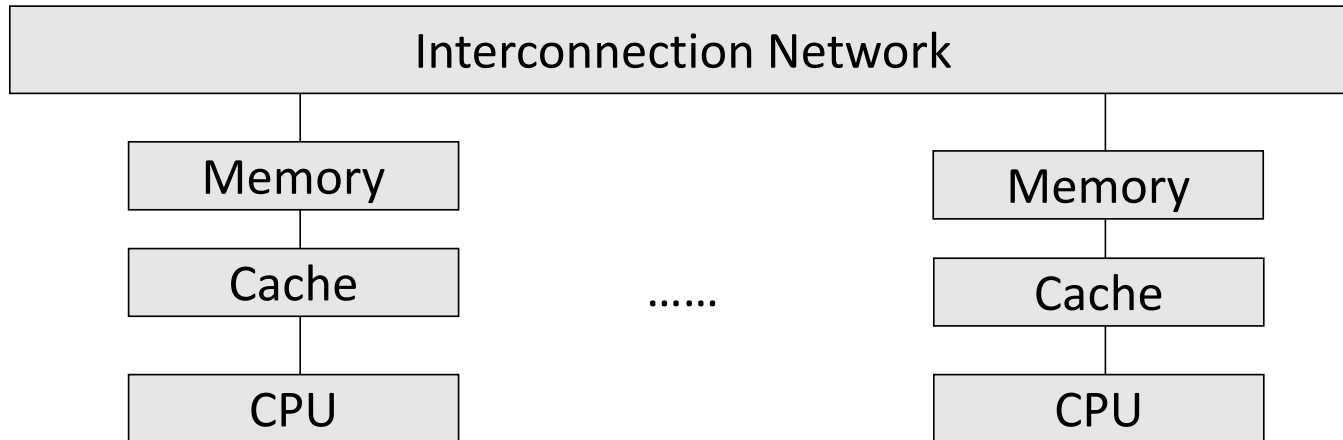
Shared-memory multiprocessors

- Processors and memory modules are connected by an interconnection network – typically a shared bus.



Distributed-memory Systems

- A distributed system
 - Has a network of computers; independent failures; no global clock
 - Communicates and coordinates only by passing messages
 - Can mean both hardware and software



Cache coherency - 2/21/21

- Caching of data in multiprocessors results in the existence of multiple copies of the same memory location in the system.
- Cache coherence allows the programmer to see the whole system as one single memory.
- When a processor modifies its local copy of the data, data inconsistency may result.
- This is known as the cache coherence problem.
- Multiprocessors with local caches need hardware and/or software support to enforce data consistency.

Solving the Cache Coherence Problem

- Don't use caches at all. 
- Single cache (L1 cache) shared by all processors.
-  Have private caches but
 - Write-through to the memory.
 - Write to an item invalidates all other copies of the item.
 - Write to an item updates all other copies of the item.

Cache coherency 协议

- Cache coherency protocols ensure that any changes made to a data item in one cache are consistently reflected across all other caches that store copies of the same data.
- When a processor writes to a memory location, the coherency scheme must guarantee the consistency of the copies.

Cache coherency protocols

- Two broad classifications
 - Write-update protocols
 - Write-invalidate protocols
 - Further classifications depending on the availability of global state information
 - Snoopy protocols
 - Directory-based protocols
- other copy*
-

Snoopy protocols

- A snoopy cache protocol relies on the hardware to monitor all writes to the memory. If a write is seen by a cache, it needs to either update or invalidate the copies that may exist elsewhere in other caches.
- Snoopy protocols can be efficiently implemented if all the processors are connected to the same shared bus, so that each cache can monitor the other caches' activity and broadcast invalidates or updates.
- In the absence of a shared bus, communication costs involved in broadcasting the write modifications become prohibitively expensive.

Directory-based protocols

- When there is no shared bus, it is more efficient to multicast the write modifications to those caches that require them.
- To do this requires storing information about which caches have copies of all cached blocks.
- A directory-based protocol takes this approach.

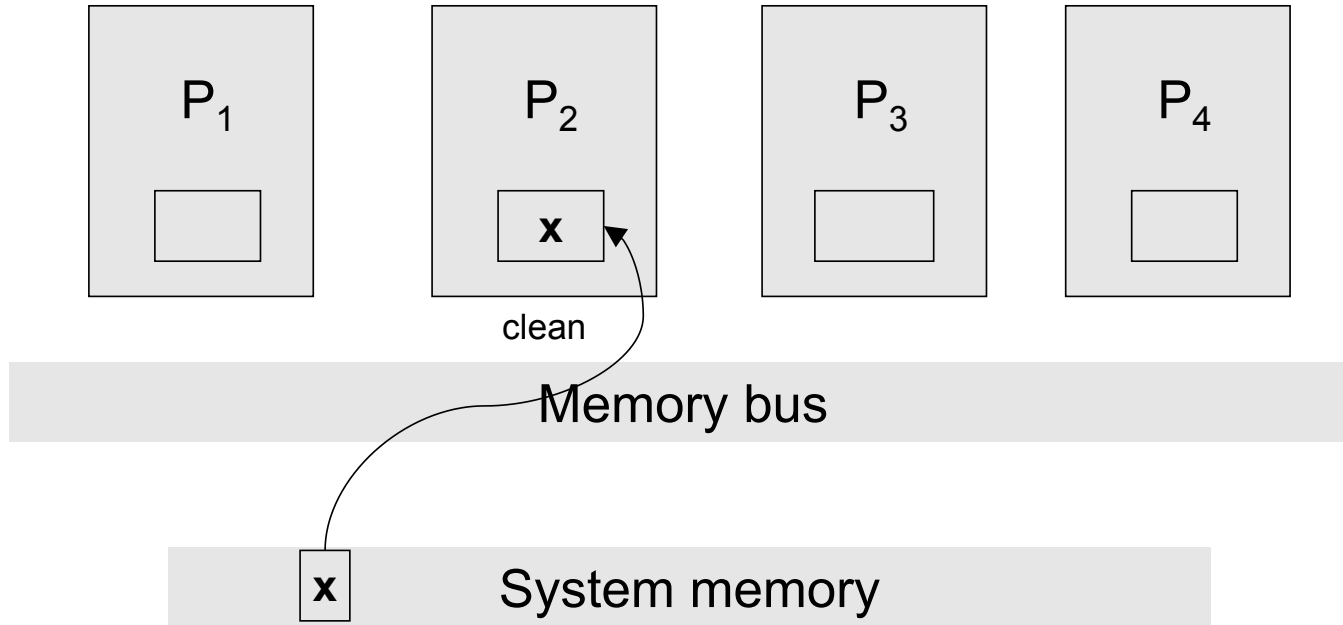
Categorizing cache misses

- **Coherency miss:** The required line was previously there in the cache, but was invalidated by another processor. There are two types:
 - True-sharing miss
 - False-sharing miss

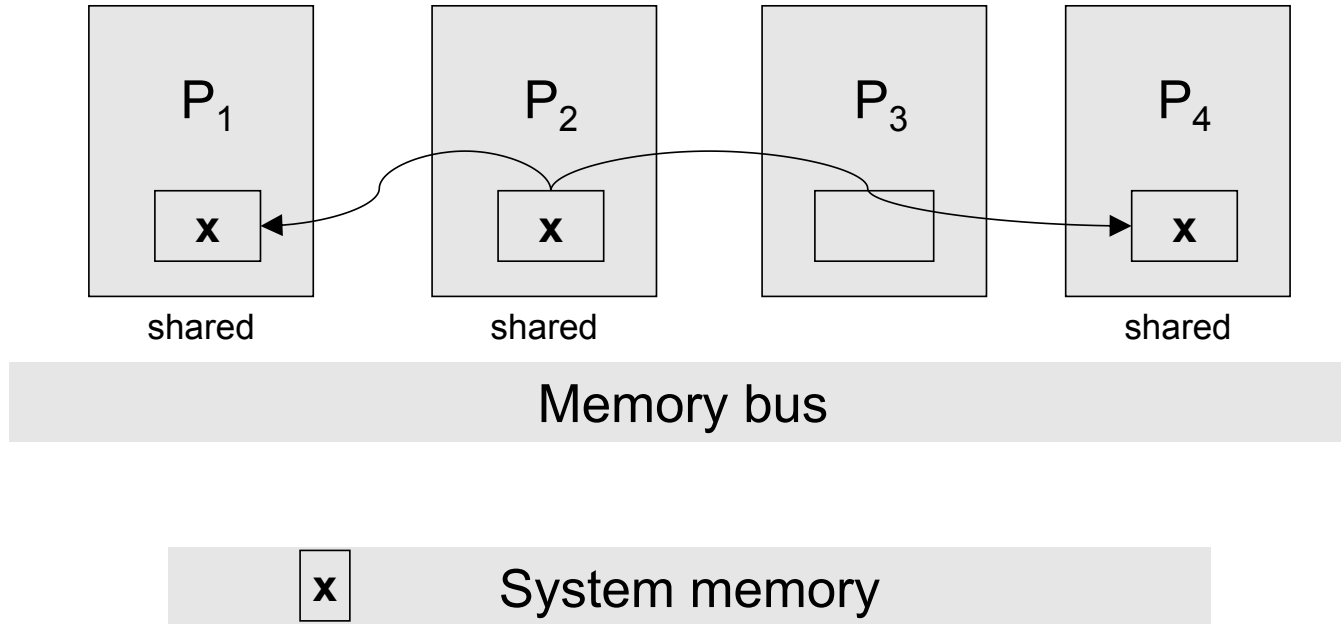
Someone invalidate you copy
↓
miss

Snoopy Protocols

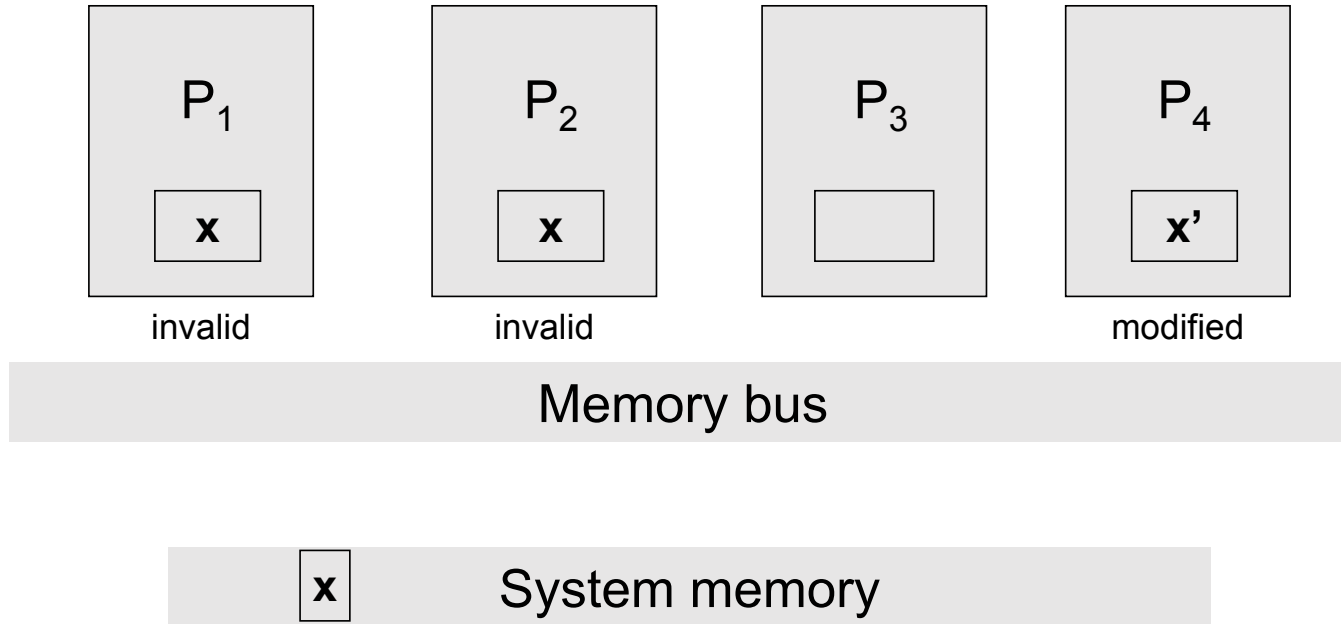
Write-invalidate protocols



Write-invalidate protocols

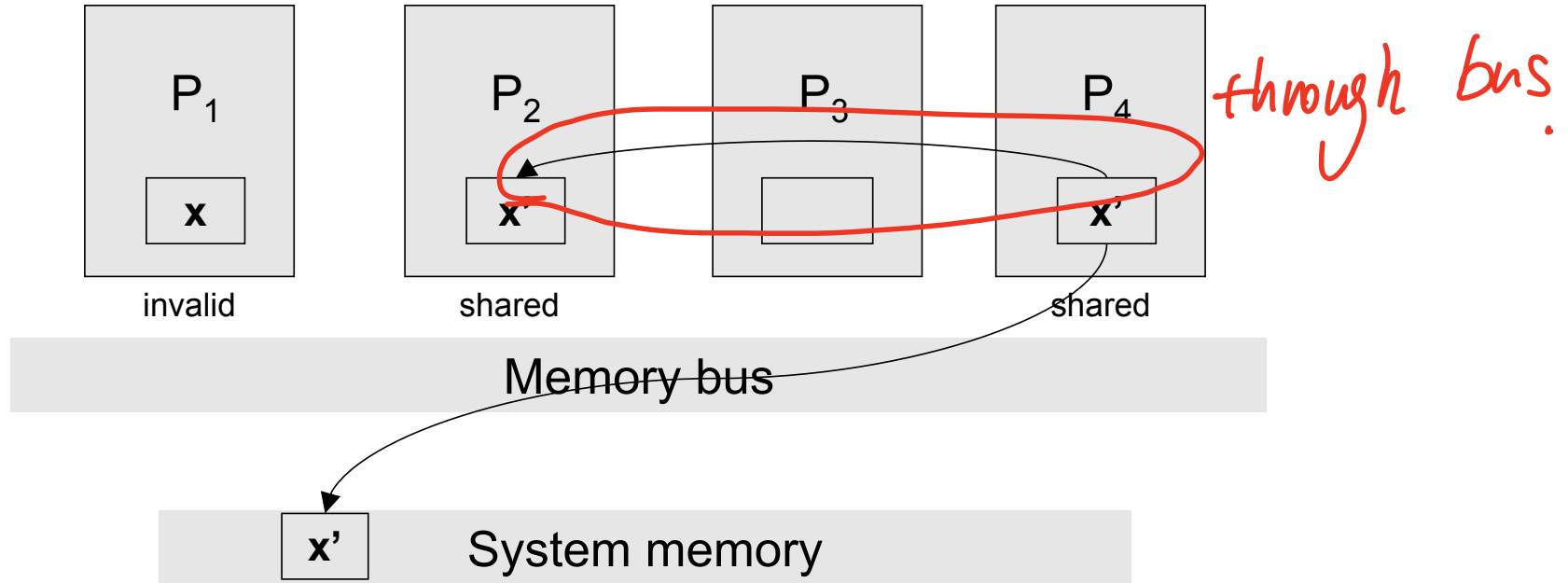


Write-invalidate protocols



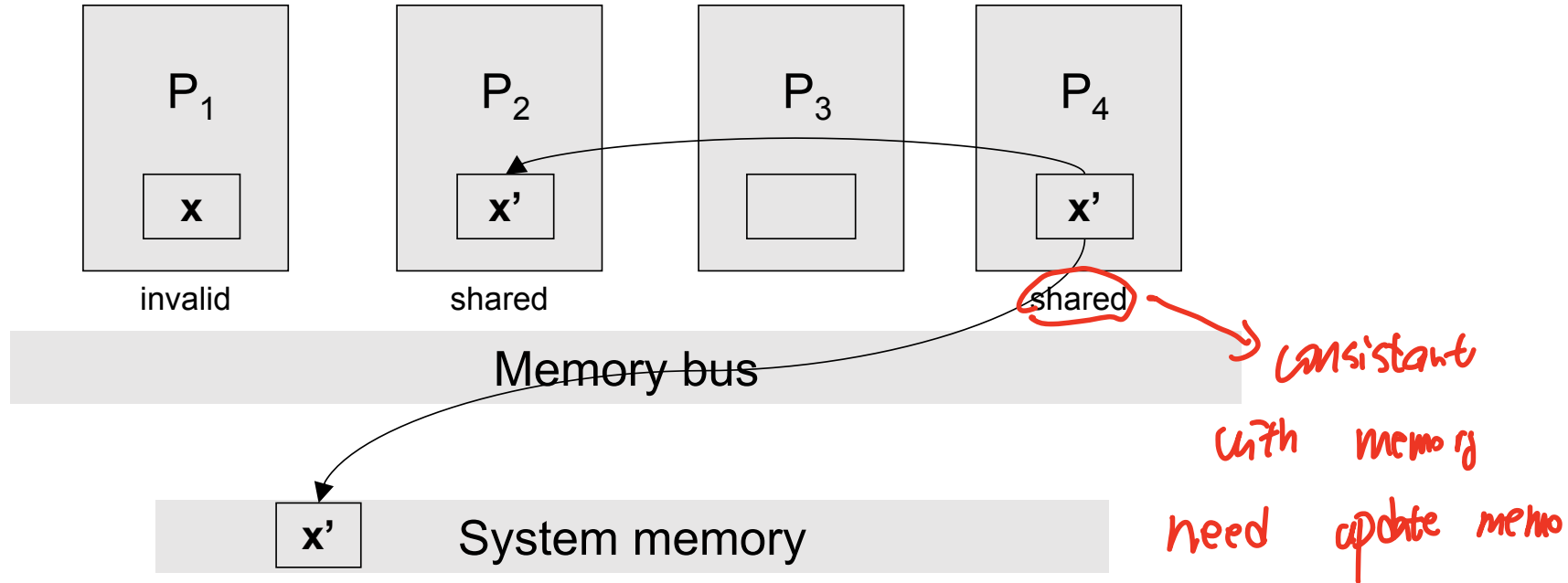
Note that P_4 's copy is more recent than what the memory has, and this has to be written back to the memory at some point.

Write-invalidate protocols



What happens if x' is not written through to the memory?

Write-invalidate protocols



What happens if x' is not written through to the memory? Answer: The state is no longer “modified”, so if it x' is not written now, it will never be.

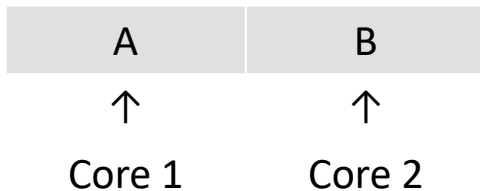
Write-invalidate protocols

- The basic write-invalidate protocol is known as the MSI protocol.
- The name comes from the status bits kept with each cache block to maintain coherence.
 - In case of MSI: M: Modified, S: Shared, and I: Invalid.
- The MOSI protocol adds another status bit – O: Owned – to MSI
- The MESI protocol adds another status bit – E: Exclusive – to MSI
- The MOESI protocol combines both MOSI and MESI.

More on these protocols later.

False Sharing

- False sharing occurs when two items that happen to map to the same cache line are used by two different processors concurrently, thus resulting in a number of cache invalidation operations.
 - Example: Item A and item B map to the same cache line. Item A is written by processor core 1 while B is written by processor core 2. Even though processor cores 1 and 2 do not share A or B, there will still be invalidation operations just because A and B map to the same cache line.



Exercise

Consider the following code fragment:

```
01  double t [8]; double a[1024][8];  // t: total
02  int i, j;
03  ....
04
05  for ( int i = 0; i < 8; ++i ) {
06      for ( int j = 0; j < 1024; ++j ) {
07          t[i] += a[j][i];
08      }
09  }
```

This code is to be executed in parallel on an 8 core system by assigning each of the outer loop iteration to a separate core. Assuming that **sizeof(double)** is 8 and the cache line size is 64 bytes, what performance problems the parallel execution may face? How can you fix these problems? Re-write the code fragment with the problems fixed.

Exercise

t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7
$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$	$a_{0,4}$	$a_{0,5}$	$a_{0,6}$	$a_{0,7}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,4}$	$a_{1,5}$	$a_{1,6}$	$a_{1,7}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$a_{2,4}$	$a_{2,5}$	$a_{2,6}$	$a_{2,7}$
...	...	...	...	...	...	...	...

Memory organized as cache lines.

Exercise

t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7
$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$	$a_{0,4}$	$a_{0,5}$	$a_{0,6}$	$a_{0,7}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,4}$	$a_{1,5}$	$a_{1,6}$	$a_{1,7}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$a_{2,4}$	$a_{2,5}$	$a_{2,6}$	$a_{2,7}$
...	...	...	...	...	...	...	...

Memory organized as cache lines.

Reducing False Sharing

- Use local (e.g., register) variables. Write to the memory just at the end.
- Reduce the cache line size. In one extreme where a cache line is no bigger than the size of the object, there won't be any false sharing. The problem is that this throws away the principle of spatial locality. This leads to a drastic increase in compulsory misses.

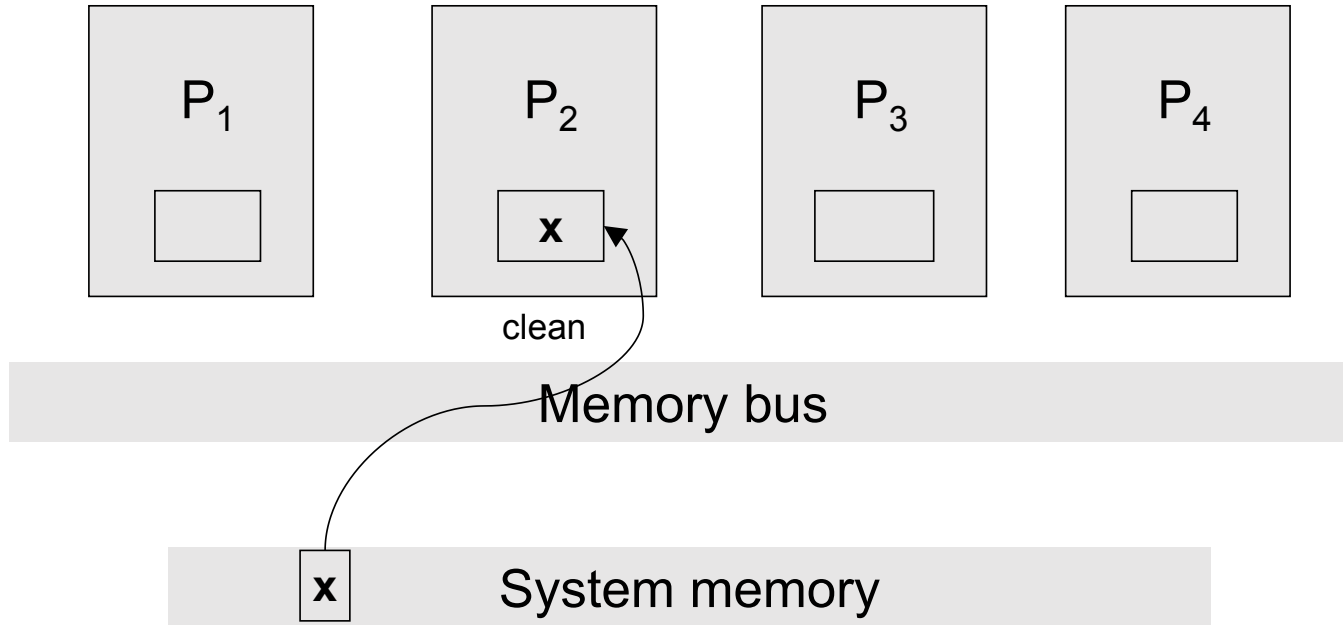
Reducing False Sharing

- Use padding to separate out objects into different cache lines. This requires analysis and identification of the false sharing, and knowledge of the size of the cache line. If the code is to be run on a different system with a different cache line size, code needs re-tuning. There is wasted memory/cache space. Padding also leads to an increase in capacity misses.

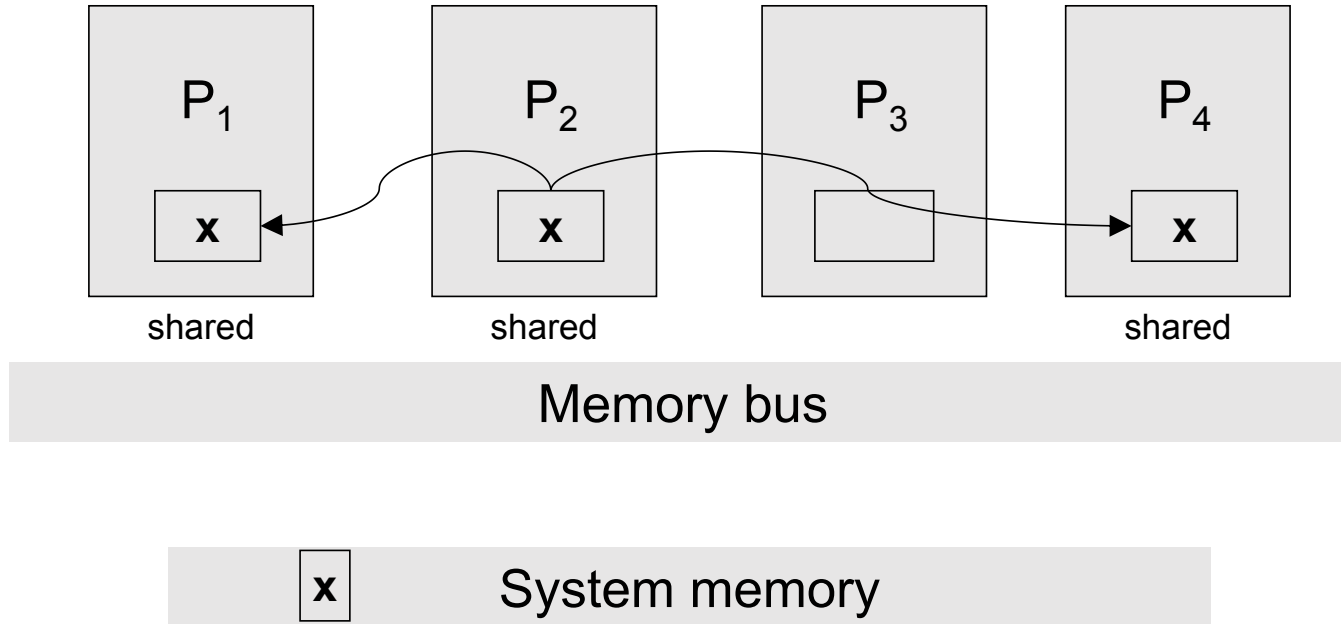
Reducing False Sharing

- With some effort, the program could be re-structured to minimize false sharing. This approach is program-specific.

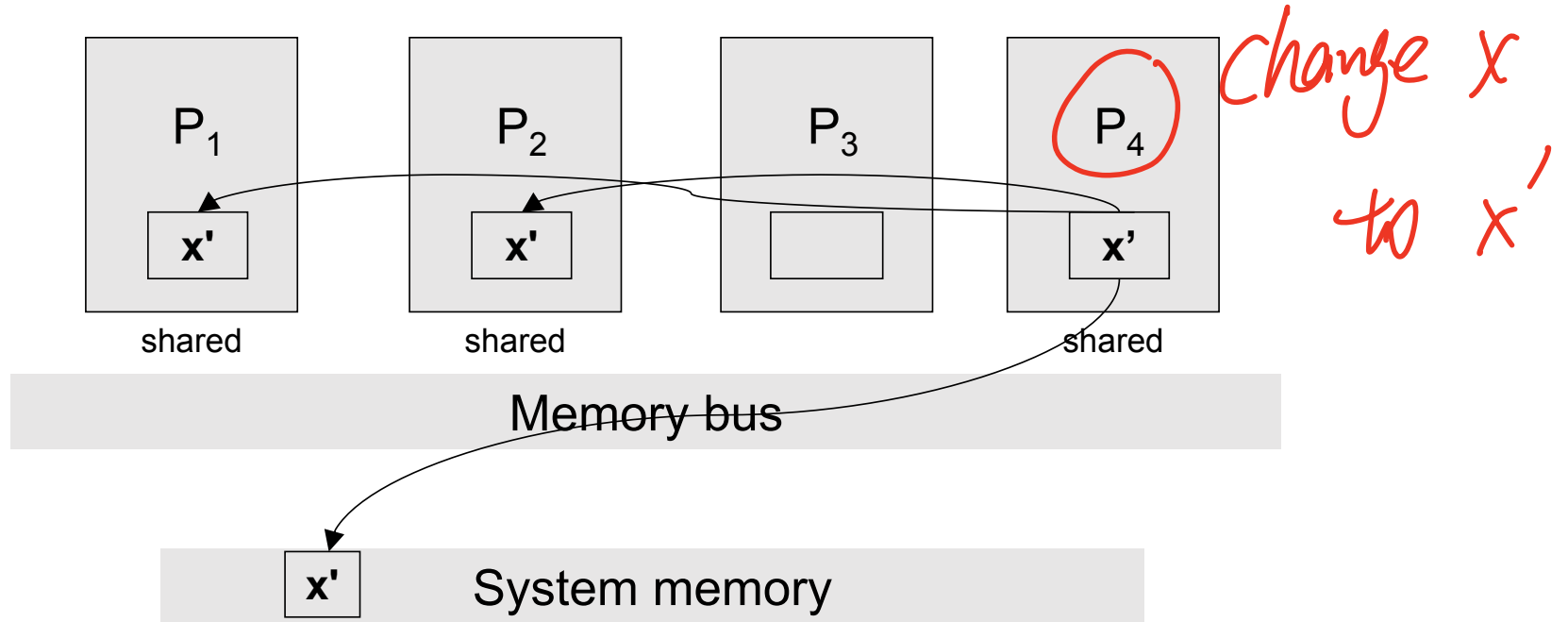
Write-update protocols



Write-update protocols

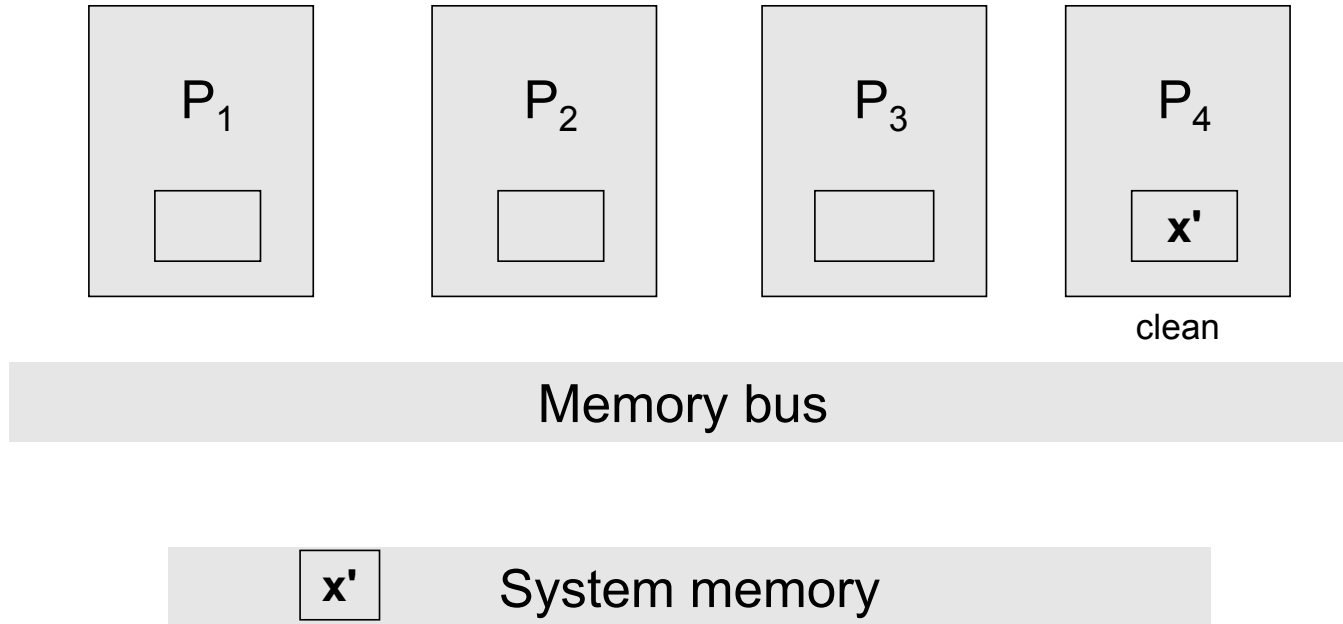


Write-update protocols

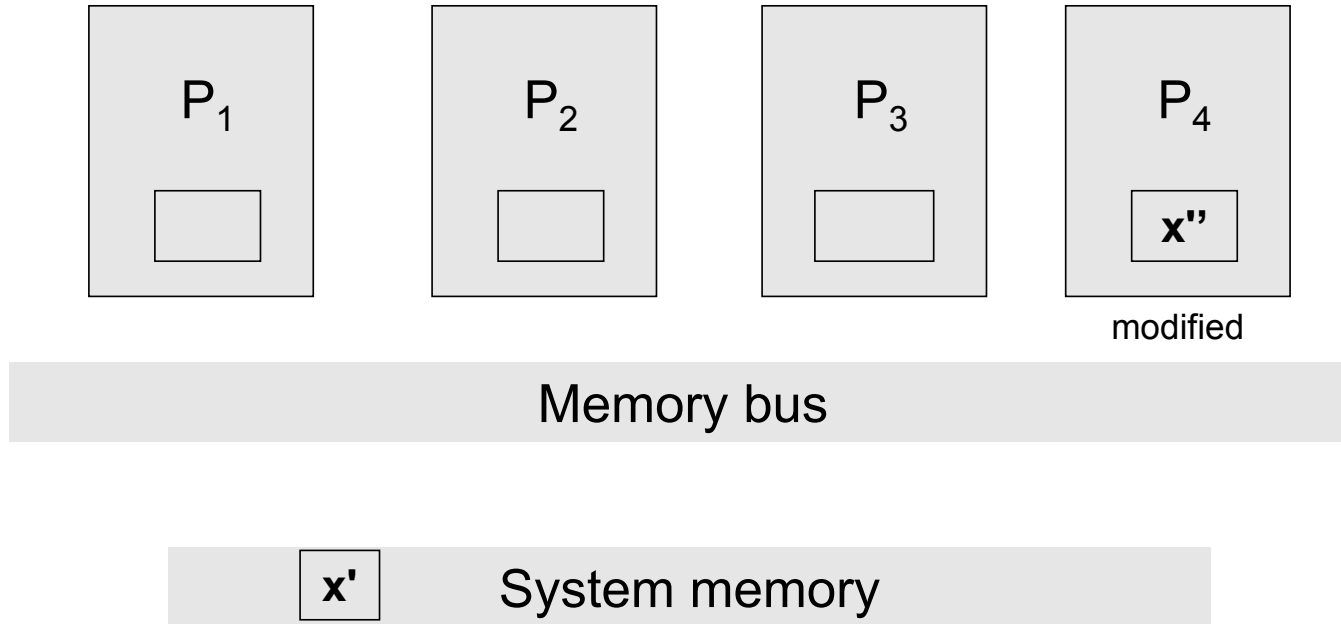


All cached copies are updated as well as the memory.

Write-update protocols

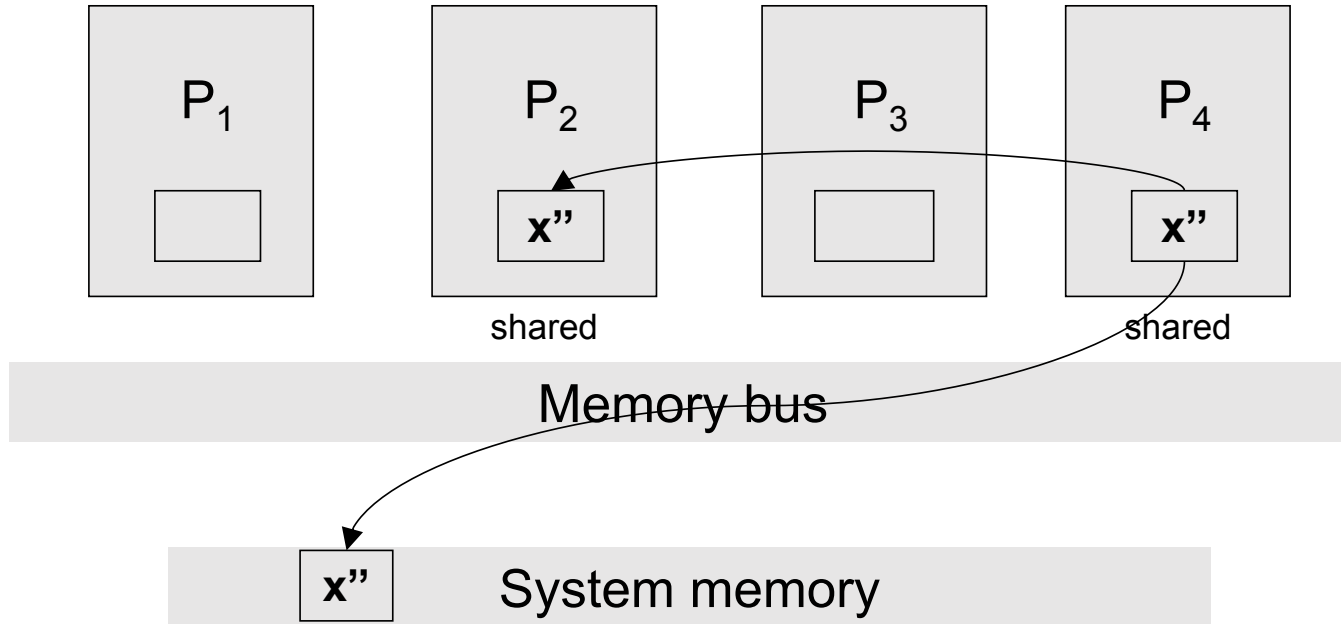


Write-update protocols



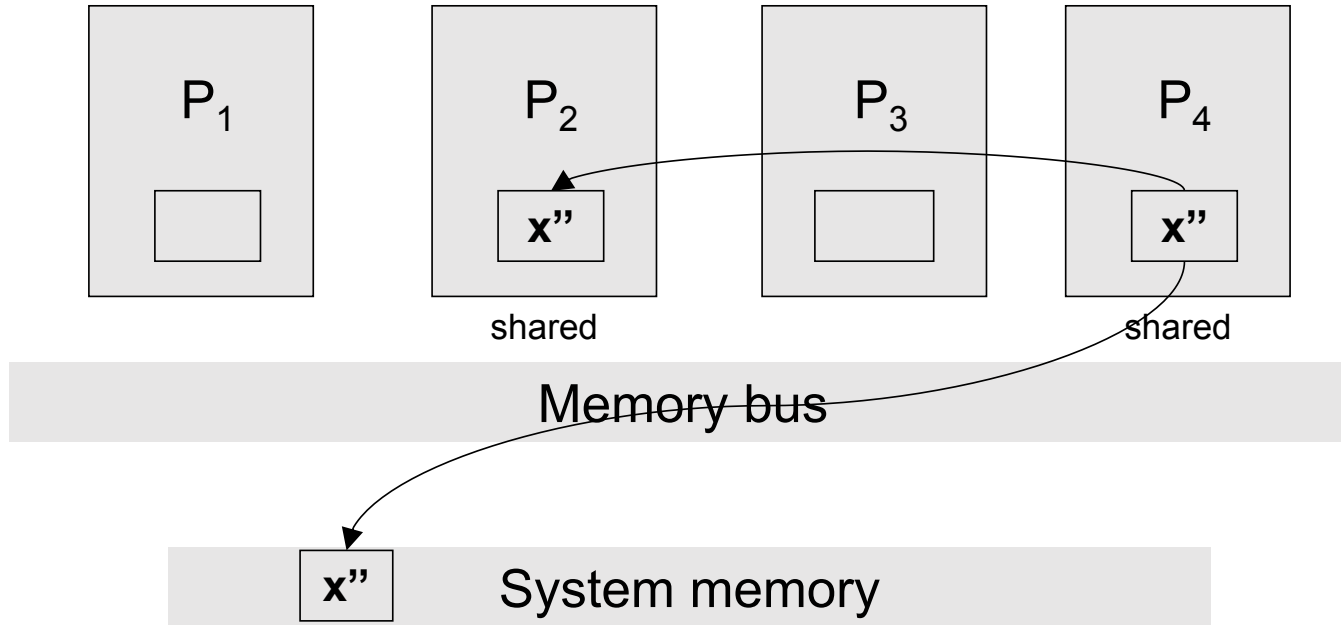
No other caches contain a copy, so P_4 holds a “modified” block (which is not written through to the memory).

Write-update protocols



What happens if x'' is not written through to the memory?

Write-update protocols



What happens if x'' is not written through to the memory? Answer: The state is no longer “modified”, so if it x'' is not written now, it will never be.

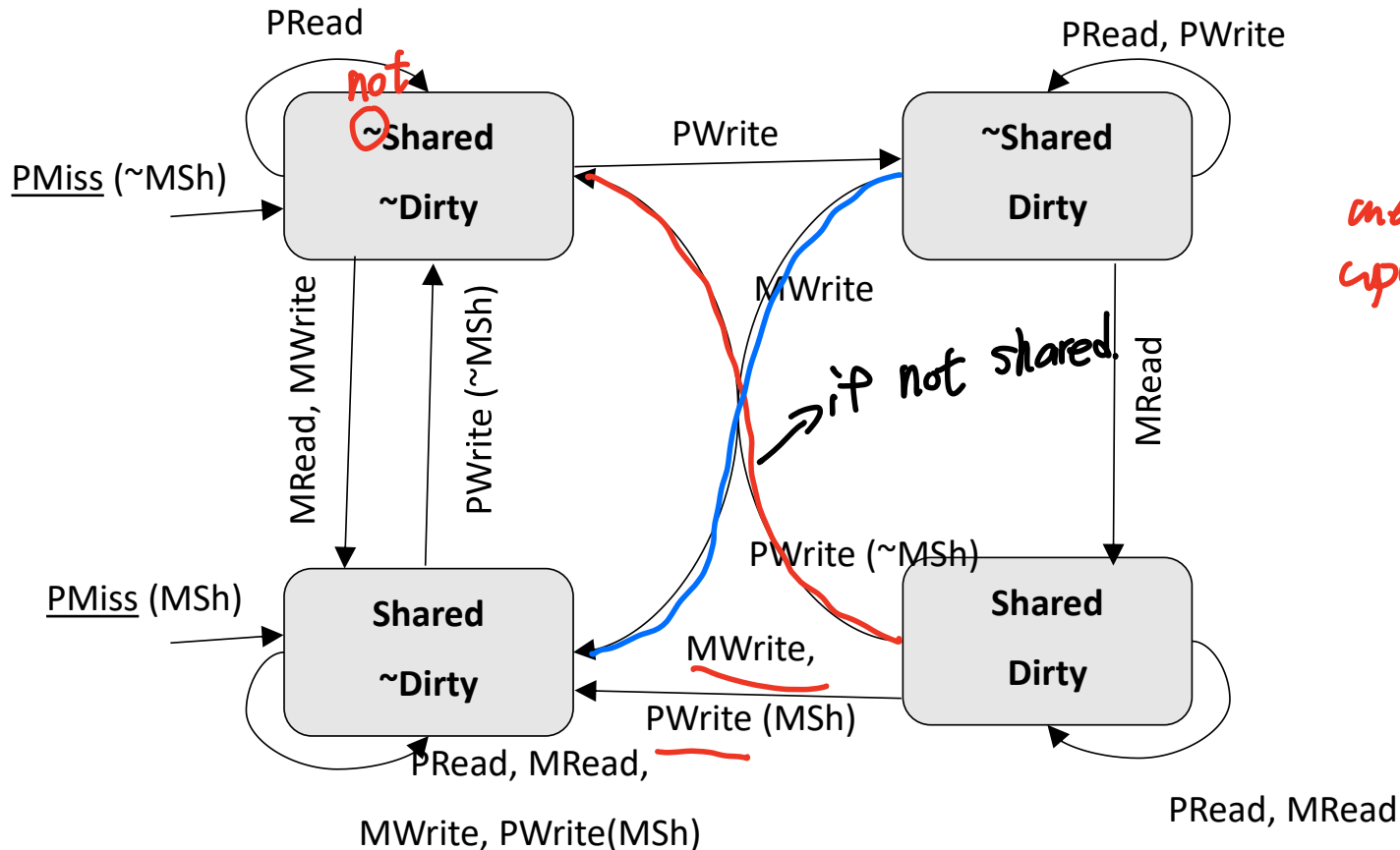
Write-update protocols

- We look at an example of a write-update protocol example (used by DEC Firefly)
 - A cache can detect when another cache shares a particular location. The bus contains a signal, *MSH*, which is asserted during a bus operation if another cache contains the requested location.
 - Each cache line contains 2 state bits: shared and dirty. There is no valid bit.
 - A write to a shared location is write-through.
 - A write to a non-shared location is write-back
- write to memory
only update cash.

Write-update protocols

- There are four states (from status bits *shared* and *dirty*). State transitions occur at these events:
 - PRead: The processor to which the cache belongs reads the location corresponding to the cache line.
 - PWrite: The processor to which the cache belongs writes to the location corresponding to the cache line.
 - MRead: A processor to which the cache does not belong reads the location corresponding to the cache line.
 - MWrite: The processor to which the cache does not belong writes to the location corresponding to the cache line.
- Transitions may depend on whether the bus signal *MSh* is active or not.

Write-update protocols



A write to a shared location is write-through.
A write to a non-shared location is write-back.

Write-update protocols

- During bootstrapping the caches are placed in a miss mode and are pre-filled with data consistent with memory. During normal operation, a line can be replaced by other lines, but is never invalidated. Having no invalidation facility poses these problems:
 - Pre-filling is a must.
 - Starting a process on a processor which had been executing some other process will incur unnecessary coherency operations on the bus.
 - Process migration (when time-sliced, etc.) will result in unnecessary coherency operations on the bus.

Write-update protocols

- The savings however are minimal: no need to check for the validity of a line (since a line is always valid), and a bit per cache line is saved.
- If the line is always valid, there will not be any false-sharing misses.

Write-invalidate protocols

- The basic write-invalidate protocol is known as the MSI protocol.
- The name comes from the status bits kept with each cache block to maintain coherence.
 - In case of MSI: M: Modified, S: Shared, and I: Invalid.
- The MOSI protocol adds another status bit – O: Owned – to MSI
- The MESI protocol adds another status bit – E: Exclusive – to MSI
- The MOESI protocol combines both MOSI and MESI.

The MSI Protocol

- The MSI protocol is a basic cache coherence protocol used in multiprocessor systems to ensure that the multiple caches maintain a consistent view of the shared memory.
- The protocol uses three distinct states to manage the status of cache lines across different caches.
 - These states are: *Modified* (M), *Shared* (S), and *Invalid* (I).
 - The *Invalid* state means a cache block is either not valid or not present.
- Each state indicates a particular status of a cache line in relation to the actions that can be performed on it and how it interacts with other caches in the system.

The M State

- The cache line is only in this cache, and it has been changed from what is in the main memory.
- This means the cache line is both exclusive to this cache and dirty (it contains data that has not been written back to memory).

The M State

- Any write operation by this cache does not need to interact with other caches or main memory.
- However, if another cache attempts to read or write this data, the current cache must write the modified data back to the main memory and change its state to Shared on a read attempt or *Invalid* on a write attempt.

The S State

- The cache line may be stored in multiple caches simultaneously and is consistent with the main memory. *not necessary*
- This state indicates that the data has not been modified in any of the caches.

The S State

- A cache can read from a cache line in the *Shared* state without any coherence action.
 - A *Shared* cache block is clean (i.e., has the same content in the memory).
- If any cache wants to write to a cache line in the *Shared* state, all other caches that have that line in the *Shared* state must invalidate their copies.
 - This ensures that the writing cache can safely change the state of the line to *Modified* without causing incoherence.

The I State

- The cache line is not valid for use; it might be stale (out of date) or initially empty.
- This state is used to denote that the cache line either needs to be fetched from memory or that it has been invalidated due to updates from other caches.

The I State

- Any attempt to read or write to a cache line in this state necessitates a coherence action:
 - Read: fetching the data from the main memory and setting the state to *Shared*.
 - Write: ensuring all other caches invalidate their copies, and setting the state to *Modified*.

Read Hit

- If a processor reads a cache line and it is either in the *Modified* or *Shared* state, it can directly read the data from the cache.

Read Miss

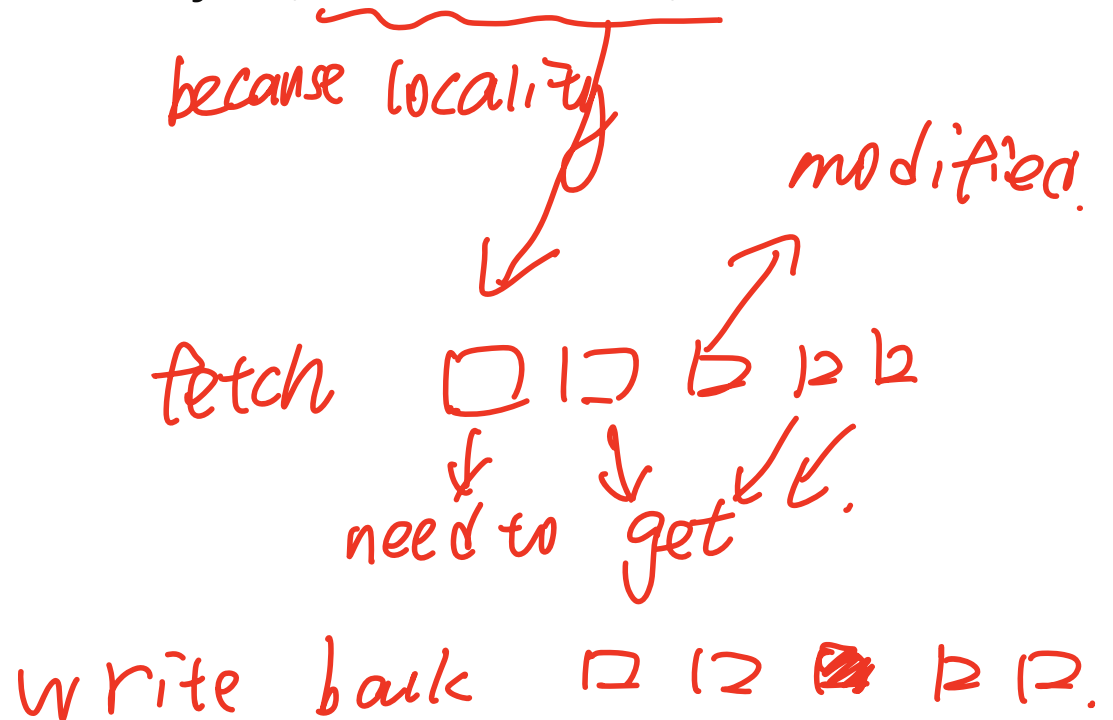
- If the cache line is *Invalid*, the cache must fetch the data from main memory or another cache.
- The cache line is set to *Shared*.
- If fetched from another cache where the line was *Modified*, the other cache must handle the write-back first.

Write Hit

- If the processor writes to a cache line that is in the *Modified* state, it proceeds with no additional coherence actions.
- If in the *Shared* state, other caches are signaled to invalidate their copies, and the state is changed to *Modified*.

Write Miss

- If the line is *Invalid*, the cache must signal other caches to invalidate if they have the line in *Shared* or *Modified*, fetch the data, and then set the line to *Modified*.



MSI Protocol: Permitted States

	M	S	I
M	X	X	✓
S	X	✓	✓
I	✓	✓	✓

The MOSI Protocol

- This adds another state, *Owned* (O), to the MSI protocol.
- The *Modified* and *Invalid* states of the MOSI protocol are as in the MSI protocol.
- The *Shared* state means the cache block is valid and shared by multiple caches (as in MSI) but the block may be more recent than the memory copy.
- The *Owned* state is a special case of the Shared state, where when the cache sees a Read request on the bus, it provides to the requester the requested data without updating memory. Also if the block needs replacement, then it needs to be written back to memory.

The MOSI Protocol

- The *Modified* state means that this cache block exists only in this cache and is dirty.
- The *Owned* state means that this cache block may exist in multiple caches and may be dirty.
 - A read miss request (from another processor) for a block in the M state in MSI will need a write-back to memory and change state to S.
 - In MOSI, we change from M to O, supply data the requester, but retain the dirty block – there is no need to write-back to memory right away.

The MOSI Protocol

- We go to the *Owned* state only from the *Modified* state when we have invalidated all other copies elsewhere.
 - At this point we know we are the owner of a dirty copy, and will supply this dirty copies to other caches when the attempt to read (upon which those caches will have the S status).
- When we have a cache line in the *Owned* state, any copies we have elsewhere in a *Shared* state will be dirty.
 - This is because the Owned cache line would have supplied this data to the caches.
- When we don't have a cache line in a *Owned* state, all copies in the *Shared* state will be clean.

MOSI Protocol: Permitted States

	M	O	S	I
M	X	X	X	✓
O	X	X	✓	✓
S	X	✓	✓	✓
I	✓	✓	✓	✓

The MESI Protocol

exclusive. → 不用 invalidate
other
copy.

- This adds another state, *Exclusive* (E), to the MSI protocol.
- The *Modified*, *Shared* and *Invalid* states of the MESI protocol are as in the MSI protocol.
 - The *Shared* state has a clean cache block matching the memory, unlike MSI which may have a dirty block.
- The *Exclusive* state means only the current cache has this block.
 - If this cache attempts to write this block, the state changes to *Modified*, and no invalidation requests are sent since no-one else has a copy.
 - If another cache attempts to read this block, the state changes to *Shared*.

MESI Protocol: Permitted States

	M	E	S	I
M	X	X	X	✓
E	X	X	X	✓
S	X	X	✓	✓
I	✓	✓	✓	✓

MOSI vs. MESI.

- MOSI can be more efficient in scenarios with frequent shared modifications, as the *Owned* state reduces the need for frequent memory write-backs.
- MESI is more efficient in scenarios where cache lines are not frequently shared between caches, as it reduces unnecessary invalidations.
- MESI requires shared data to be clean, while MOSI does not.
 - The owner (O) in MOSI is responsible for writing back to the memory when the block is evicted.

MOSI plus MESI

- The MOESI protocols combines both MOSI and MESI.
- Like MOSI, the *Shared* state may be dirty (i.e., may have a more recent copy of data than memory).

MOESI Protocol: Permitted States

	M	O	E	S	I
M	X	X	X	X	✓
O	X	X	X	✓	✓
E	X	X	X	X	✓
S	X	✓	X	✓	✓
I	✓	✓	✓	✓	✓

The Forward (F) State

- In the MESI protocol, when there is a read request to a cache block that exists in the Shared state in multiple caches, the read request could be serviced by:
 - The memory (slow), or
 - All the caches that hold the block in the Shared state (too many responses sent to the requester)
- The *Forward* state is a special *Shared* state that is responsible for servicing read requests, eliminating the need for multiple caches to respond to read requests. 冗余数据
- Note that *Forward* is a clean state (as opposed to *Owned* which is dirty).

Snoopy protocols

- A snoopy cache protocol relies on the hardware to monitor all writes to the memory. If a write is seen by a cache, it needs to either update or invalidate the copies that may exist elsewhere in other caches.
- Snoopy protocols can be efficiently implemented if all the processors are connected to the same shared bus, so that each cache can monitor the other caches' activity and broadcast invalidates or updates.
- In the absence of a shared bus, communication costs involved in broadcasting the write modifications become prohibitively expensive.

Directory-based protocols

- When there is no shared bus, it is more efficient to multicast the write modifications to those caches that require them.
- To do this requires storing information about which caches have copies of all cached blocks.
- A directory-based protocol takes this approach.

Directory-based protocols

- A directory keeps the status of all cache lines. Cache operations consult this directory as required.
- The directory can be centrally kept, or can be distributed.
- A central directory scheme is easy to implement but it may become a bottleneck. A distributed directory is therefore a better choice.

Directory-based protocols

- Directory-based protocols need to keep quite a lot of state information and thus require a fair amount of storage.
- Yet, these protocols are more scalable than snoopy protocols.

Directory

- Each cache block has an entry in the directory
- The entry has information about which caches have the corresponding cache block and whether or not the block might be dirty in some of the caches.
 - 1 bit per cache to denote which caches contain the block
 - 1 dirty bit
- Cache states are the same, but instead of snooping on a shared bus, we are using a directory to ensure coherence.

¿?